SENTINEL SAS v4.1

ENTERPRISE CORE ENGINE & REFERENCE SCHEMA

Lead Architect: **Muhammad Ahmad Atif**
Affiliation: **IMSC University**
Context: **BS AI (Semester 4)**

System Baseline: **Multi-Domain Architecture**
Target Engines: **MySQL 8.0+ / MariaDB 10.6+**
Date Stamped: **June 2026**

1. ARCHITECTURAL OVERVIEW

The **SENTINEL SAS v4.1** system represents a multi-domain, triple-role enterprise management console engineered around a **Hash-First Indexing System**. Its purpose is to mitigate duplicate-refund arbitrage, enforce transaction immutability, audit vendor quality metrics, and control discrete access clearances (Customer, Employee, and Owner) within hardware-bound execution loops.

The core cryptographic backbone relies on a 16-character composite hexadecimal key format that uniquely tracks an individual physical inventory item from manufacturing to end-user retail checkout.

The Anti-Arbitrage 16-Character Hash Breakdown

Supplier ID	Item Type	Serial Signature	Member Segment
[0 - 3]	[4 - 7]	[8 - 11]	[12 - 15]
Hex (4 Chars)	Hex (4 Chars)	Hex (4 Chars)	Hex (4 Chars)

2. DOMAIN DEFINITIONS & ROLE HIERARCHIES

ACCESS DOMAIN	SYSTEM ROLE	FUNCTIONAL RESPONSIBILITIES & CLEARED DATA PATHS
Consumer Domain Layer	`Customer`	Permitted to read public pricing data, view explicit user purchase history records, check remaining subscription periods, and verify membership tier multipliers. Blocked from arbitrary backend lookups or manual ledger overrides.
Operational Hub	`Employee`	Granted terminal-node access to split and decode composite keys into structural components. Authorized to file hardware fault reports via the Defect Logger and trace serial runs inside terminal bounds.
Administrative Core	`Owner`	Global administrative supervisor. Cleared to observe system-wide net revenue metrics, track real-time security logs, and impose permanent data restrictions upon anomalous supplier nodes dropping below the 85% integrity threshold.

3. COMPREHENSIVE TABLE MATRIX

3.1 Table: `SYSTEM_USER`

The core authentication center tracking global identity profiles across all three administrative domains.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`userUID`	VARCHAR(50)	`PRIMARY KEY`	Unique alphanumeric tracking string assigned to each unique system identity.
`username`	VARCHAR(50)	`UNIQUE` / NOT NULL	Human-readable handle selected by or allocated to the user during registration.
`biometricKeyHash`	CHAR(64)	NOT NULL	SHA-256 secure hash mapping the operator or user's biometric registration passphrase.
`systemRole`	VARCHAR(15)	NOT NULL	Enforces value distribution explicitly matched to: `'Customer'`, `'Employee'`, or `'Owner'`.
`accountStatus`	VARCHAR(15)	DEFAULT 'Active'	State monitoring rule: `'Active'`, `'Suspended'`, or `'Locked'`. See `SP_RegisterFailedLogin`.
`isFirstLogin`	BOOLEAN	DEFAULT TRUE	Triggers hardware registration workflows upon an initial workstation validation flag.
`failedLoginAttempts`	TINYINT UNSIGNED	DEFAULT 0	Counter mechanism clamping limits up to 10. Automatically triggers an account lock at 5 consecutive violations.

3.2 Table: `SUPPLIER`

Monitors manufacturing partner channels and automates pipeline isolation via live integrity updates.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`supplierID`	CHAR(4)	`PRIMARY KEY`	Hexadecimal value string (0-9, A-F) acting as characters 0-3 of the asset composite fingerprint.
`name`	VARCHAR(100)	NOT NULL	Corporate manufacturing identity name.
`integrityScore`	DECIMAL(5,2)	DEFAULT 100.00	Dynamic evaluation rating calculated via total defect logging rates. Floor: 0.00, Ceiling: 100.00.
`vendorStatus`	VARCHAR(15)	DEFAULT 'Active'	

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
			Conditional flag distribution: `'Active'`, `'Restricted'`, or `'Suspended'`. Falling under 85% enforces 'Restricted'.

3.3 Table: `ITEM_TYPE`

Maps character positions 4–7 of the structural key layout to absolute product categorization strings.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`typeCode`	CHAR(4)	PRIMARY KEY	Hexadecimal code signature representing classifications (e.g., `'99B1'` for Solid-State Storage Units).
`categoryName`	VARCHAR(100)	NOT NULL	Descriptive name indicating item cluster categorization definitions.

3.4 Table: `PRODUCT`

The master data definitions tracking active products. Maps a unique 12-character structural run string.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`unitHash_12`	CHAR(12)	PRIMARY KEY	Aligned structural string joining `supplierID` (4) + `typeCode` (4) + `serialSig` (4).
`supplierID`	CHAR(4)	FOREIGN KEY	References `SUPPLIER(supplierID)`. Structural rule: must equal characters 0–3 of `unitHash_12`.
`typeCode`	CHAR(4)	FOREIGN KEY	References `ITEM_TYPE(typeCode)`. Structural rule: must equal characters 4–7 of `unitHash_12`.
`basePrice`	DECIMAL(10,2)	NOT NULL	Standard catalog retail value in base currency prior to applying customer tier percentage reductions.

3.5 Table: `STOCK_LEDGER`

Monitors hardware intake workflows and logs low-level structural defects discovered inside storage zones.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`ledgerID`	VARCHAR(50)	PRIMARY KEY	Unique audit sequence code tracking stock processing instances.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`unitHash_12`	CHAR(12)	FOREIGN KEY	References `PRODUCT(unitHash_12)`. Identifies the explicit hardware run signature profile.
`isDefective`	BOOLEAN	DEFAULT FALSE	Boolean identifier. When switched true, fires automated triggers adjusting master vendor indices.
`severity`	VARCHAR(10)	NULLABLE	Enforces value constraints limiting data strings to: `'Minor'`, `'Major'`, or `'Critical'`.
`loggedByUID`	VARCHAR(50)	FOREIGN KEY	References `SYSTEM_USER(userUID)`. Links accountability tracking to specific terminal operators.

3.6 Table: `CUSTOMER_PROFILE`

Maintains specific billing data, current wallet assets, and mapping arrays for discrete member hex assignments.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`customerID`	CHAR(4)	PRIMARY KEY	Alphanumeric hex string corresponding directly to characters 12–15 of the final purchase hash.
`userUID`	VARCHAR(50)	FOREIGN KEY	References `SYSTEM_USER(userUID)` with a strong relational 1-to-1 linkage constraint.
`tierType`	VARCHAR(10)	FOREIGN KEY	References `TIER_DISCOUNT(tierType)`. Controls calculation dynamics inside checkout procedures.
`accountBalance`	DECIMAL(10,2)	DEFAULT 0.00	Available digital wallet credit reserves held inside client operational storage partitions.

3.7 Table: `EMPLOYEE_PROFILE`

Binds workspace specialists to physical terminal nodes and records operator action logs.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`employeeID`	VARCHAR(50)	PRIMARY KEY	Internal organizational key profile identifying operational workshop agents.
`userUID`	VARCHAR(50)	FOREIGN KEY	

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
			References `SYSTEM_USER(userUID)`. Maps directly into shared core security segments.
`designation`	VARCHAR(50)	NOT NULL	Value mapping restricted strictly to: `'Desk Agent'`, `'Shift Supervisor'`, `'Systems Specialist'`.
`terminalNodeBinding`	VARCHAR(100)	NULLABLE	Binds account credentials to explicit hardware identifiers during node verification procedures.

3.8 Table: `TRANSACTION`

The core structural header mapping processing metadata, baseline balances, and terminal records.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`transactionID`	VARCHAR(50)	PRIMARY KEY	Alphanumeric string generated to uniquely verify specific ledger events.
`employeeID`	VARCHAR(50)	FOREIGN KEY	References `EMPLOYEE_PROFILE(employeeID)` to track structural processing configurations.
`customerID`	CHAR(4)	FOREIGN KEY	References `CUSTOMER_PROFILE(customerID)`. Nullable if processing checkout items for anonymous walk-ins.
`finalNetTotal`	DECIMAL(10,2)	NOT NULL	True financial total. Enforces constraint checking alignment: `finalNetTotal = rawTotal - discountApplied`.

3.9 Table: `HASH_LOG`

The anti-arbitrage validation ledger. Evaluates structural layout allocations during transaction requests.

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
`fullHash_16`	CHAR(16)	PRIMARY KEY	The absolute 16-character hexadecimal composite identity string of an explicit physical item.
`transactionID`	VARCHAR(50)	FOREIGN KEY	References `TRANSACTION(transactionID)`. Ties items directly to financial audit events.
`status`	VARCHAR(15)		

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION & BUSINESS RULES
		DEFAULT 'Sold'	State monitoring configuration rules: `'Sold'`, `'Returned'`, or `'Flagged'`. Duplicate records auto-force 'Flagged'.
`flagReason`	VARCHAR(255)	NULLABLE	Textual string generated automatically when security loops flag structural violations or double spending events.

4. DATABASE AUTOMATION MECHANICS

Sentinel SAS v4.1 enforces schema integrity via state-driven table automation. The absolute definitions of these database interactions prevent application logic errors from circumventing system policies.

4.1 Automated Integrity Computations

When an Employee creates an entry in `STOCK_LEDGER` marked with `isDefective = TRUE`, an internal system trigger executes immediately. It scans the aggregate product logs for the relative `supplierID` (extracted dynamically from the 12-character base code), recalculates the running defect-to-supply ratio, and establishes a fresh `integrityScore`.

If this score scales under **85.00%**, the vendor status is transitioned to `'Restricted'`. This status flag blocks future item validations for that supplier code across the system network, as implemented in the schema definition:

```
CREATE TRIGGER TR_StockLedger_UpdateSupplierIntegrity
AFTER INSERT ON STOCK_LEDGER FOR EACH ROW ...
```

4.2 Arbitrage Deflection Mechanics

To neutralize double-refund arbitrage exploits (where malicious actors attempt to submit duplicate claims against identical serial records), the system implements a pre-insertion hook on `HASH_LOG`.

Before an event commits, the engine performs an index search across active clusters. If a identical 16-character fingerprint is found marked as `'Sold'`, the transaction state is rejected, rewritten into an internal system intercept event, flagged for security auditing, and committed into a monitored audit state along with precise context descriptors.

5. ADVANCED CLAUDE REFINEMENT BLUEPRINT

The following design specification is constructed to allow downstream Large Language Models (specifically Claude 3.5 Sonnet) to ingest this structural specification and generate optimized application logic blocks matching the schema design exactly.

[SYSTEM SYSTEM DESIGN SPECIFICATION CONTEXT INGESTION BLUEPRINT]

Act as a principal database administrator and systems engineer specializing in secure enterprise-tier architectures. You are assigned to implement a high-throughput backend infrastructure for the SENTINEL SAS v4.1 Engine.

Core Framework Input Constraints:
1. Multi-Domain Structure: Data paths must validate access levels based on `SYSTEM_USER.systemRole` boundaries (Customer, Employee, Owner).
2. Crucial 16-Char Composition Rule: Every physical item key maps precisely to a 16-character hexadecimal validation block. Your generation structures must isolate characters 0-3 as Supplier ID, 4-7 as Item Classification Type, 8-11 as Batch Serial Run, and 12-15 as Customer Segment Verification Suffix.
3. Structural Cascades: When generating API routes or query optimization layers, adhere precisely to the foreign keys matching table references (e.g., `PRODUCT.unitHash_12` linking directly across to operational stock files).

Explicit Engineering Deliverables Required:
- Deliver highly robust, exception-guarded transaction management blocks matching the execution pattern of `SP_ProcessTransaction`.
- Produce connection layer configurations validating structural hash rules via regular expression patterns before committing parameters to database targets.
- Emplace strict token validation routines checking that execution routes labeled under specific roles cannot execute cross-domain procedures.

Process the schema data provided in this reference documentation to ensure 100% data definition compatibility. Avoid stub code blocks or incomplete error logic.